CSCI 532

Kody Rogers
Cody Smith
Nicholas Gouldin
Rochester Institute of Technology

1 Bibliography and Patent Search

1.1 FL Feature Analysis vs Centralized Learning

1.1.1 Overview of FL and Centralized Learning

Definition of Centralized Learning

Centralized Learning (CL) is a traditional machine learning scheme [7]. Data to train or build a model is gathered and stored by a single central server or cloud [7]. This involves aggregated data collection from multiple devices, primarily user data. Clients then access the model by sending requests to the server [7].

Definition of Federated Learning

Federated Learning (FL) is a machine learning scheme that seeks to preserve access to user data while maintaining the privacy of individual users [8]. Client devices collaboratively train a single model, coordinated by a central server, while keeping their local data on-device [8]. This approach creates a decentralized form of machine learning [8].

Centralized Learning:

All raw data is collected from various sources and devices (e.g., mobile devices, sensors, institutions) and uploaded to a server or cloud. The server trains the model on a unified dataset. Clients are then granted access by querying the server [7].

Federated Learning:

Client devices keep their raw data local. Instead of sending data, client devices train the server's model locally, sending model updates (gradients/weights) to the server. The server aggregates these into a global model and returns it to the clients to complete a round of training[8].

Key Differences

- **Data locality:** For centralized learning, all data is transferred to and stored in a central server. For Federated Learning, data remains local on clients, only model updates are shared [7], [8].
- **Architecture:** For centralized learning, a single central server manages both data storage and model training. For Federated Learning, the Central Server coordinates training, but the computation is distributed across clients [8].
- **Privacy:** For centralized learning, there is a big privacy risk. Raw data is being collected into one location, which could be either intercepted during its transfer or harvested fully by attackers [7]. Federated learning is much better for privacy, because raw data never leaves client devices. Updates sent to the central server could be reverse-engineered for data, but measures exist to make this process very difficult for attackers [8].

1.1.2 Five FL Aggregation Techniques

Overview of aggregation's role in federated learning

After a training round has taken place in federated learning, each device will produce a sequence of tensors. Each tensor is a matrix, representing the biases or weights of an individual layer of the device's local machine learning model. These tensors are then sent to the centralized server. However, there are separate models on each device, each with their own different tensors. To handle the differing models, we need to find a way to combine the tensors together to collectively produce a new central model [7, 8].

FedAvg

FedAvg is a standard aggregation algorithm in FL, introduced by McMahan et al. [3]. The server computes a weighted average of each parameter across clients. The weight of a client's parameters solely depends on how many samples the client has trained with, making those clients more impactful than the others.

Algorithm

$$w^{(t+1)} = \frac{\sum_{k=1}^{K} n_k \, w_k^{(t+1)}}{\sum_{k=1}^{K} n_k}$$

Variables:

- t = training round
- $w^{(t+1)}$ = new global model parameters (tensors)
- K = number of participating clients
- $w_k^{(t+1)}$ = local parameters of client k
- n_k = number of data samples client k trained on

5 Factors:

- **Efficiency:** Really fast, cuts down communication by sending updates once per round, not once per batch.
- **Robustness:** No issues if a client fails to participate mid training. The server just averages who participates. Highly vulnerable to malicious devices, as they can introduce a large amount of data and get strong influence over the model.
- **Scalability:** Easily scaled, just a linear computation of weights.
- **Privacy:** Good, only weights and parameters are shared. However, other techniques might be needed as sending model weights can leak info.
- **Accuracy:** As good as centralized training with IID data, struggles with non-IID data.
- **Heterogeneity:** Handles when clients have a different amount of data, but fails to acknowledge when the distribution of labels are probabilistically skewed.

Krum

Krum is a Byzantine-robust algorithm introduced by Blanchard et al. [4]. It selects a singular client that it deems the most "trustworthy" of all clients every round. The trustworthy client is the

only client that the server processes an update from. Krum does this to become resilient against malicious attacks, through a form of a robustness called **Byzantine Robustness**. Byzantine robustness accounts for the possibility of a Byzantine failure, of where a client can behave arbitrarily. A learning algorithm with this property still converges even if **up to f** clients send bad or adversarial updates every round.

Algorithm

- Assume a client update is a vector, which is the flattened gradient (weight matrix) of the parameters. Calculate the pairwise euclidean distances between each client update. A single client update u_i is compared to each other update sent u_j.
- Distance formula:

$$D_{ij} = \left\|u_i - u_j\right\|_2^2, \quad \forall i \neq j$$

- For each client i, sort the euclidean distances from least to greatest. After sorting, select the set N_i which contains the (n - f - 2) closest other clients.
- From the N_i set we assign an individual Krum score to the client. A client's Krum score is the sum of the (n - f- 2) closest other client distances.

$$S(i) = \sum_{j \in N_i} D_{ij}$$

- The client with the smallest krum score is chosen. The server only processes the update of this chosen client.

Variables:

- n = number of client updates sent to server
- f = maximum number of malicious clients (hyperparameter, assumed to be true)
- u_i, u_j = client update pair
- N_i = Set of closest clients by distance
- D_{ij} = Client pair euclidean distance

Trimmed Mean

The Trimmed Mean algorithm is an aggregation scheme proposed by Yin et al. [5].Similar to the Krum algorithm, the Trimmed Mean algorithm aims to achieve **Byzantine Robustness** in federated learning. The Trimmed Mean algorithm protects against malicious updates by discarding the most extreme coordinate values in each dimension of the gradient vectors. The remaining values in each update are then averaged out for a final result.

Consider an update vector x1, x2, x3. This vector is considered to have 3 coordinates i.e dimensions.

3 client example:
100, 2, 4, (malicious)
5, 2, 1, (normal)
8, 3, 5, (normal)

in coordinate x1, 100 would be discarded as an extreme value, giving the algorithm some resilience.

Algorithm

- For each dimension k = 1, … d, collect all the values from each update u and put them into individual lists

$$\{u_1^{(k)}, u_2^{(k)}, \ldots, u_n^{(k)}\}.$$

- Sort the dimension vectors in ascending order.
- Trim the dimension vectors by removing the largest and smallest f values (assuming there are f malicious clients)
- Compute the mean of the remaining n - 2f values for each coordinate vector.

$$g^{(k)} = \frac{1}{n-2f} \sum_{i \in I_k} u_i^{(k)},$$

  Ik is the set of values after trimming the list
  k is the coordinate index, not an exponent
- The global update to the model is then set as:
  g = (g1, g2, … gn)

Variables:

- n = total number of clients
- f = max. number of byzantine clients assumed (hyper param)
- d = dimension count of update vector
- I_k = set of values after the trimming of dimension vectors
- i = ith coordinate value from the trimmed vectors (ith client)
- g = final global aggregated update vector

FedProx

Fedprox serves as a solution to handling non-IID data. It was introduced by Li et al. as an extension of FedAvg to handle heterogeneous data [6]. It does this by adding a proximal term to discourage individual clients from straying too far from the global model. As a result, FedProx is great at handling clients with heterogeneous data and/or resources.

Algorithm

- Each client must solve a modified local problem. The client must find the minimum of the corresponding objective function:

$$\min_w \; F_i(w) + \frac{\mu}{2}\|w - w^{(t)}\|^2.$$

- Server sends a current global model wt to selected clients.
- Each client updates its model by minimizing the objective function. The objective function is usually minimized using stochastic gradient descent (SGD) or an optimizer such as Adam.
- Clients send their updated parameters $w_i^(t+1)$ back to the server.
- Server aggregates the updates weighted by their local dataset sizes.

$$w^{(t+1)} = \sum_{i=1}^{n} \frac{n_i}{N} w_i^{(t+1)}, \quad N = \sum_i n_i.$$

Variables:

- Fi(w) is the local loss for the dataset training round
- The rest of the equation is a proximal term, which penalizes the client quadratically from straying farther away from the global model.
- w = client model weights (optimization target)
- w(t) = global model weights (can't change)
- $x - x(t)$ is the difference between the weights of the server and the client

- μ = hyperparameter to determine the strength of the misalignment penalty

MedMedian

FedMedian is another Byzantine-robust algorithm [5]. The algorithm runs on a simple concept, which is to take the median values of coordinate vectors across all clients rather than the mean.
Algorithm

- For each coordinate k, produce a vector V(k) that represents the coordinate values across all clients.

$$V^{(k)} = \{u_1[k], u_2[k], \ldots, u_n[k]\}$$

- Sort the coordinate vectors from least to greatest values.
- Select a median. Let n be the dimension count of the vector. If n is odd:

$$g[k] = V^{(k)}_{\frac{n+1}{2}}$$

if n is even:

$$g[k] = \frac{1}{2}\left(V^{(k)}_{n/2} + V^{(k)}_{n/2+1}\right)$$

g = the value of the coordinate in our final server weight vector
g[k] = kth value of the server weight vector, with k = current dimension
- The final vector is assembled:

$$g = (g[1], g[2], \ldots, g[d])$$

1.1.3 Comparative Feature Table

- efficiency, robustness, scalability, privacy, accuracy, heterogeneity
- Time complexities are given for efficiency, with being the clients and d being the dimension count of the flattened weight matrix. For FedProx, E = local epoch count and b = batch size.
- Sources: [3–6]

See Table 1 for the comparative features of aggregation methods.

1.1.4 Summary

- **FedAvg:** serves as the most basic algorithm, giving minimal computation requirements. This makes FedAvg very fast but it requires IID data and assumes the clients are fully honest and controlled. Use FedAvg for maximum speed in an optimal scenario for data and clients [3].
- **FedProx:** handles heterogenous data, but has no Byzantine defense. It can be used for when clients are trustworthy but computational power and/or dataset sizes vary greatly[6].
- **Trimmed Mean:** is Byzantine Robust, with good accuracy. Use Trimmed Mean when you want a good balance of security and accuracy [5].
- **FedMedian:** is robust, with a fairly simple implementation. Use FedMedian if you want a simple implementation that is robust, while being able to afford some accuracy loss [5].
- **Krum:** is the most secure algorithm of the 5, but has a very poor quadratic time complexity. Use Krum if security is paramount in training and when the client count is modest [4].

1.2 Privacy and Security in Federated Learning

1.2.1 How FL Addresses Privacy Concerns

- **Keeping data local so raw data never leaves client devices.**
A fundamental advantage of federated learning lies within its privacy. Raw data never leaves a client device, which significantly reduces the risk of data leaks during transmission or storage [6, 7].
- **Secure Aggregation:**
Secure aggregation: is a special technique that allows a server to collaboratively compute the sum of client update values without revealing the values themselves. This is achieved by using MPC-style masking/cryptography. Production-grade protocols can tolerate clients dropping out and are efficient at scale [8], and the concept is reinforced by official patents describing the technique [21].
- **The process of Homomorphic Encryption (HE):**
HE allows for the computation of encrypted updates. This is highly useful for if the client can't fully entrust the server to perform secure computations on the updates they send.
- **Differential privacy (DP):**
DP ensures that model updates reveal very little about individual users by adding calibrated noise. DP can be applied at the client or server level, offering a guarantee of privacy at the cost of some accuracy. Patents describe mechanisms for implementing DP in ML pipelines [24, 25].
- **Trusted Execution Environments (TEE):**
TEEs, such as Intel Software Guard Extensions (SGX), provide secure enclaves for sensitive computations. Research has shown FL integration with SGX [12]. In addition, there are patents that describe confidential processing pipelines [26]. TEEs reduce server-side risk for data leaks but also require the hardware vendor (Intel, ARM) to implement the enclave correctly [13].

1.2.2 Vulnerabilities and Threats

- **Model Poisoning / Backdoor Attacks (sending poisoned updates)**
Malicious clients manipulate their updates so that the model appears to behave normally, but will misclassify attacker chosen triggers. These attacks can bypass poor defenses and are persistent across training rounds [15].
- Data Poisoning Attacks (manipulating local training data)
Attackers modify their own training data and assign incorrect labels. Data may be labeled to degrade a model's accuracy or shift the model towards a specific bias. Without proper measures (such as Byzantine Robust Algorithms), malicious clients can significantly damage a global model's performance [16].
- Inference Attacks:
Inference attacks exploit the leaked information found from the gradients, weights and/or global model to reverse engineer private training data.
There are two types of inference attacks:
- Membership inference: Adversaries infer whether a specific record was a part of training [16]. For example, a model

Table 1: Comparative Feature Table

Aggregation	Efficiency	Robustness	Scalability	Privacy	Accuracy	Heterogeneity
FedAvg	Very Fast ($O(nd)$)	Not Robust	High	Neutral	High (IID)	Weak
Krum	Slow ($O(n^2d)$)	Best	Limited	Neutral	Lower	Weak
Trimmed Mean	Moderate ($O(nd \log n)$)	Strong	Reasonable	Neutral	Good	Indirect
FedProx	Fast ($O(Ebd)$)	Not Robust	High	Neutral	High	Fully Capable
FedMedian	Moderate ($O(nd \log n)$)	Strong	Reasonable	Neutral	Lower	Indirect

could be trained on cancer patient data. If an attacker can infer that a user was part of the server's dataset, they now know that the user likely has cancer.

- Gradient Inversion: Reconstructing private inputs from shared updates through techniques such as Deep Leakage. Deep Leakage is capable of recovering sensitive images and labels [17].

1.2.3 *Mitigation and Defense Strategies*

- **Explaining Robust Aggregation Methods (Krum, Trimmed Mean) that defend against poisoned updates**
  - Krum / Multi-Krum: selects an update that is closest to all other updates by euclidean distance, to a max of f malicious clients [19].
- **Anomaly Detection for identifying suspicious client behavior**
  - Server-side defenses must analyze updates to flag suspicious clients, either by finding outliers or down-weighting clients with overly similar updates [21].
- **Enhanced Encryption Techniques (Homomorphic encryption, maybe or maybe not secure multiparty computation)**
  - Secure Aggregation: practical and scalable [8], with patents supporting federated aggregation [21].
  - Homomorphic Encryption: Enables the aggregation and computation of client updates in an encrypted form [10, 11], with patents supporting it [22, 23].
  - TEEs: Creates an enclave environment where data is protected [12, 13], with training patents [26].
- **Differential Privacy and Privacy Budgeting**
  - DP ensures privacy of user data by applying clipping and noise to updates. A privacy budget is the idea of applying clipping and noise based on a budget for each client. Once the budget for a client is exhausted through the noising and clipping of updates, privacy is no longer guaranteed for the client. Patents formalize these mechanisms [24, 25].

1.2.4 Summary FL improves privacy by keeping raw data local but remains vulnerable to poisoning and inference attacks. A secure FL system requires combining aggregation defenses, anomaly detection, and encryption, though challenges remain.

2 Data Preparation Strategy

2.1 Dataset

- MedMNIST is the dataset that we are planning on using for the duration of this project. It contains a collection of medical images that are pretty well organized and easy to work with. This makes it perfect for the scope of this project.

2.2 Data Cleaning

- Blank Image Removal: Images with zero pixel intensity values were removed to avoid training on empty samples.
- Duplicate Removal: Utilized the np.unique() function to remove all identical samples, increasing data quality and reducing the bias of the future training model.

2.3 Data Normalization

- Pixel values were normalized from a $[0, 255]$ range to $[0, 1]$ to stabilize model training and increase the rate of convergence.

2.4 Data Augmentation

- We plan to use data augmentation to help prevent model overfitting, and make the model more robust to potential image variations. Such augmentations would include rotations, flips, and zooms of images within the dataset.

2.5 Data Partitioning

- Training data is partitioned into 20 dummy clients, with 5 samples per label per client.
- This simulates a federated learning scenario where a small group of clients contributes with a small, localized dataset.
- in convert_clients.py, each client dataset is saved as a .pt file (pytorch format) and as individual .png images with class labels.
- Data Partitioning was implemented in convert_clients.py, using the partition_clients() and save_client_data() functions.

2.6 Data Labeling

- Used a label map (`LABEL_MAP`) in `convert_clients.py` to verify that the dataset labels match the intended class definitions in MNIST.

```
LABEL_MAP = {
    0: "Adipose tissue",
    1: "Background",
    2: "Debris",
    3: "Lymphocytes",
    4: "Mucus",
    5: "Smooth muscle",
    6: "Normal colon mucosa",
    7: "Cancer-associated stroma",
    8: "Colorectal adenocarcinoma epithelium"
}
```

2.7 Data Corruption

- We plan to implement data corruption in part 3 of the project via Personal Information Deficiency (PID). A subset of client datasets will have their labels intentionally altered to simulate malicious clients and unreliable data sources. We will alter the datasets in two ways:
- Systematic mislabeling: for example, all samples of a class (2: Debris) might be relabeled as another (5: Smooth Muscle).
- Randomized mislabeling: 15% of the samples are assigned incorrect labels at random.

3 Formulating Initial Model

For the model, we will be training a Neural Network from scratch.

3.1 Goals Of Model

- The model has to be compact such that it can be accurate when working with small images. The images of the dataset are only 28x28 pixels.
- The model should train quickly without overfitting.
- The model should be easily extensible to federated per-client training (small model state for frequent parameter exchange).
- The model should have a practical training recipe and augmentations.

3.2 Model Characteristics

- Kaiming Initialization: Other weight initialization methods tend to cause gradients to either vanish or explode out of control, such as Xavier initialization. Kaiming initialization aims to prevent this and cause smoother gradients.
- Stacked 3x3 Convolutions: This size of convolution is good for optimize both accuracy and efficiency. Stacking two 3x3 convolutions gives an effective receptive field of 5×5 while using fewer parameters, making it more efficient.
- Batch Normalization: Using batch normalization stabilizes and speeds up training. It slightly regularizes, improving generalization on small datasets.
- Two-convolution Pattern: Going through two convolutions before downsampling preserves spatial information and lets the block learn richer features per spatial scale.
- Max Pool Downsampling: After each block, the spatial dimension gradually decreases, such that later layers operate on more compact feature maps. In this model, it will decrease from a starting 28 to 14 to 7. This will reduce computational cost and reduce translational variance.
- Moderate Network Width: The network width will be 32 -> 64 -> 128. This network size will balance capacity and overfitting.
- Weight Decay: In combination with dropout and augmentations, they give robust regularization for the patches where overfitting is a risk.

Overall, the characteristics of the model make it efficient and accurate for small images such as the ones in the MedMNIST dataset. It also has features to reduce the cost of running the simulation and prevent the model from overfitting.

References

[1] L. Li, Y. Fan, M. Tse, and K.-Y. Lin, "A review of applications in federated learning," *Computers & Industrial Engineering*, vol. 149, p. 106854, Nov. 2020.

[2] S. Abdulrahman, H. Tout, H. Ould-Slimane, A. Mourad, C. Talhi, and M. Guizani, "A Survey on Federated Learning: The Journey From Centralized to Distributed On-Site Learning and Beyond," *IEEE Internet of Things Journal*, vol. 8, no. 7, pp. 5476–5497, Apr. 2021.

[3] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-Efficient Learning of Deep Networks from Decentralized Data," in *Proc. AISTATS*, 2017, pp. 1273–1282.

[4] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, "Machine Learning with Adversaries: Byzantine Tolerant Gradient Descent," in *Proc. NeurIPS*, 2017, pp. 119–129.

[5] D. Yin, Y. Chen, R. Kannan, and P. Bartlett, "Byzantine-Robust Distributed Learning: Towards Optimal Statistical Rates," in *Proc. ICML*, 2018, pp. 5650–5659.

[6] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated Optimization in Heterogeneous Networks," in *Proc. MLSys*, 2020.

[7] S. Abdulrahman *et al.*, "A Survey on Federated Learning: The Journey From Centralized to Distributed On-Site Learning and Beyond," *IEEE Internet of Things Journal*, vol. 8, no. 7, pp. 5476–5497, 2021. [Online]. Available: icaangroup.com

[8] L. Li *et al.*, "A review of applications in federated learning," *Computers & Industrial Engineering*, vol. 149, p. 106854, 2020. [Online]. Available: ScienceDirect

[9] K. Bonawitz *et al.*, "Practical Secure Aggregation for Privacy-Preserving Machine Learning," in *Proc. ACM CCS*, 2017. [Online]. Available: ACM Digital Library

[10] M. Mansouri *et al.*, "SoK: Secure Aggregation Based on Cryptographic Schemes for Federated Learning," in *Proc. Privacy Enhancing Technologies (PoPETS)*, 2023. [Online]. Available: petsymposium.org

[11] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic Encryption for Arithmetic of Approximate Numbers (CKKS)," in *ASIACRYPT*, 2017, pp. 409–437. [Online]. Available: IACR

[12] N. Dowlin *et al.*, "CryptoNets: Applying Neural Networks to Encrypted Data with High Throughput and Accuracy," in *Proc. ICML*, 2016, pp. 201–210. [Online]. Available: Proceedings of Machine Learning Research

[13] T. Kim *et al.*, "Secure Federated Learning in TensorFlow Using Intel SGX," *arXiv:2106.10143*, 2021. [Online]. Available: VISA Research Lab

[14] M. Song, H. Wang, and C. Wang, "Confidential Federated Learning," *Communications of the ACM*, vol. 67, no. 4, pp. 94–103, 2024. [Online]. Available: Communications of the ACM

[15] E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, and V. Shmatikov, "How to Backdoor Federated Learning," in *Proc. AISTATS*, 2020. [Online]. Available: Proceedings of Machine Learning Research

[16] V. Tolpegin, S. Truex, M. E. Gursoy, and L. Liu, "Data Poisoning Attacks Against Federated Learning Systems," in *Proc. ESORICS*, 2020. [Online]. Available: ACM Digital Library

[17] M. Nasr, R. Shokri, and A. Houmansadr, "Comprehensive Privacy Analysis of Deep Learning: Passive and Active White-box Inference Attacks against Centralized and Federated Learning," in *Proc. IEEE Symposium on Security and Privacy*, 2019. [Online]. Available: arXiv

[18] L. Zhu, Z. Liu, and S. Han, "Deep Leakage from Gradients," in *Proc. NeurIPS*, 2019. [Online]. Available: NeurIPS Proceedings

[19] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, "Machine Learning with Adversaries: Byzantine Tolerant Gradient Descent," in *Proc. NeurIPS*, 2017. [Online]. Available: NeurIPS Papers

[20] D. Yin, Y. Chen, R. Kannan, and P. Bartlett, "Byzantine-Robust Distributed Learning: Towards Optimal Statistical Rates," in *Proc. ICML*, 2018. [Online]. Available: arXiv

[21] C. Fung, C. J. M. Yoon, and I. Beschastnikh, "Mitigating Sybils in Federated Learning Poisoning (FoolsGold)," in *Proc. RAID*, 2019. [Online]. Available: USENIX

Patents:

[22] "Secure aggregation of information using federated learning," U.S. Patent US20220374544A1, 2022. [Online]. Available: Google Patents

[23] C. Gentry and S. Halevi, "Homomorphic evaluation including key switching, modulus switching, and dynamic noise management," U.S. Patent US9608817B2, Mar. 28, 2017. [Online]. Available: Google Patents

[24] "Fully Homomorphic Encryption," U.S. Patent US9083526B2, 2015. [Online]. Available: Google Patents

[25] F. McSherry *et al.*, "Differential data privacy," U.S. Patent US7698250B2, Apr. 13, 2010. [Online]. Available: Google Patents

[26] E. D. Friedman *et al.*, "Differential privacy for message text content mining," U.S. Patent US11290411B2, Mar. 29, 2022. [Online]. Available: Justia Patents Search

[27] F. Yan *et al.*, "Verifiable Deep Learning Training Service," U.S. Patent US11,816,575 B2, Nov. 14, 2023. [Online]. Available: Patent Images